

TripGenie - Documentation Technique Détaillée (MVP Finale)

Étape 3 : Spécifications & Architecture Auteur : Alexis Loties (Projet Solo - Certification)

1. User Stories & Maquettes (Priorisation MoSCoW)

Must Have (Essentiel)

- Story 1** : En tant qu'utilisateur, je veux dialoguer naturellement avec un concierge IA pour définir mes envies de voyage (destinations, ambiance, budget), afin de ne pas remplir de longs formulaires complexes.
- Story 2** : En tant qu'utilisateur, je veux un itinéraire précis basé sur des données web réelles (Vols, Hôtels, Activités), afin d'avoir une estimation fiable du coût et de la faisabilité de mon séjour.
- Story 3** : En tant qu'utilisateur, je veux pouvoir m'inscrire et me connecter, afin de conserver mes voyages dans un espace personnel sécurisé.

Should Have (Important)

- Story 4** : En tant qu'utilisateur, je veux voir la météo en direct et des photos HD de ma destination, pour mieux me projeter visuellement dans mon futur voyage.
- Story 5** : En tant qu'utilisateur, je veux pouvoir voter pour mes éléments préférés (hôtels ou activités) au sein d'un pack, afin de garder une trace de mes coups de cœur.
- Story 6** : En tant qu'utilisateur, je veux pouvoir consulter l'historique de tous mes voyages générés, afin de retrouver une ancienne planification en un clic.

Could Have (Bonus)

- Story 7** : En tant qu'utilisateur, je veux pouvoir partager mon itinéraire via WhatsApp, afin de coordonner facilement mon projet de voyage avec mes proches.
- Story 8** : En tant qu'utilisateur, je veux pouvoir basculer entre le mode sombre et le mode clair, afin d'adapter l'interface à mon confort visuel et à mon environnement.
- Story 9** : En tant qu'utilisateur, je veux une interface fluide et "responsive", afin de pouvoir consulter mes détails de voyage confortablement sur mon smartphone.
- Story 10** : En tant qu'utilisateur, je veux voir les liens officiels des établissements suggérés, afin de pouvoir effectuer mes réservations finales en toute confiance.

2. Architecture du Système (MVP)

Diagramme de Haut Niveau & Flux de Données

```
graph TD
    User((Utilisateur)) <--> Frontend[React / Vite]
    Frontend <--> Backend[Node.js / Express]

    %% Services Internes
    Backend <--> DB[(Supabase / PostgreSQL)]
    Backend <--> Auth[Supabase Auth / JWT]

    %% API Externes
```

```
Backend <--> AI[Anthropic - Claude 3.5]
Backend <--> Search[Tavily - Web Search Agent]
Backend <--> Photos[Unsplash API - Photos HD]
Backend <--> Payments[Stripe API - Réservations]
```

%% Flux internes complexes

```
AI -. -> |Demande de données fraîches| Search
Search -. -> |Retourne Vols/Hôtels/Météo| AI
```

Description du Flux de Données

- Interaction Utilisateur** : L'utilisateur envoie une requête via le `Frontend`.
- Orchestration Backend** : Le `Backend` vérifie l'identité via `Supabase Auth`.
- Intelligence Agentique** : Le `Backend` sollicite `Claude 3.5` qui orchestre une recherche via `Tavily` pour obtenir des données réelles (météo, vols, hôtels).
- Enrichissement Visuel** : Le `Backend` récupère des visuels via `Unsplash` pour chaque destination.
- Persistance** : Le pack final est stocké dans la `DB` (`Supabase`) pour consultation ultérieure.
- Transaction (Optionnel)** : Le flux se termine par une session `Stripe` pour la validation de la réservation.

3. Conception Technique (Composants & Classes)

Architecture des Classes (Backend - Node.js)

Le backend est structuré en services modulaires. Voici les classes clés qui pilotent la logique métier :

```
classDiagram
    class AIService {
        +assemblePack(webData: Object) : Object
        +chatIntake(message: String) : String
        +generateItinerary(prompt: String) : String
    }
    class SearchAgent {
        +searchFlights(query: String) : Array
        +searchHotels(query: String) : Array
        +searchEvents(query: String) : Array
    }
    class DBService {
        +saveTrip(userId: UUID, data: Object) : Promise
        +getTrips(userId: UUID) : Array
    }

    AIService --> SearchAgent : "Récupère les données réelles"
    AIService --> DBService : "Persistance des packs"
```

- AIService** : C'est le cerveau de l'application. Sa méthode `assemblePack` fusionne les données web avec le contexte utilisateur.
- SearchAgent** : Gère l'interface avec Tavily AI. Ses méthodes extraient les meilleurs prix et disponibilités en temps réel.

- **DBService** : Gère les interactions avec Supabase pour la lecture/écriture sécurisée des données.

Composants de l'Interface (Frontend - React)

L'UI est découpée en composants autonomes pour garantir une expérience fluide :

- **ChatWidget** : Interface de dialogue. Il capture les besoins de l'utilisateur et affiche les réponses de l'IA.
- **ResultsView** : Orchestre l'affichage du pack voyage. Il utilise des "sub-components" pour rendre les cartes d'hôtels et de vols.
- **TripCard** : Affiche les détails HD d'une suggestion (Unsplash) et gère l'état local des votes utilisateur.
- **AuthLayout** : Gère les formulaires de connexion/inscription via le provider Supabase.

Conception de la Base de Données (Modèle Entité-Relation)

Le choix de **PostgreSQL** a été fait pour garantir l'intégrité des données et la flexibilité du stockage des itinéraires via le type `JSONB`.

```
erDiagram
    USER ||--o{ TRIP : "organise"
    USER ||--o{ VOTE : "émet"
    TRIP ||--o{ VOTE : "reçoit"

    USER {
        uuid id PK
        string email
        string password_hash
        timestamp created_at
    }

    TRIP {
        uuid id PK
        uuid user_id FK
        string destination
        jsonb pack_data "Contenu de l'itinéraire IA"
        timestamp created_at
    }

    VOTE {
        bigint id PK
        uuid user_id FK
        uuid trip_id FK
        string item_id "ID de l'hôtel ou activité"
        string type "hotel | activity"
        timestamp created_at
    }
```

4. Diagrammes de Séquence (Flux Principaux)

Cas d'Usage 1 : Génération d'un Itinéraire Signature (IA Agentique)

```
sequenceDiagram
    participant U as Utilisateur
    participant F as Frontend
    participant B as Backend
    participant T as Tavily Agent
    participant C as Claude AI

    U->>F: Soumet son besoin (ex: "Monaco VIP")
    F->>B: POST /api/ai/generate
    B->>T: Recherche Web (Météo, Hôtels, Vols)
    T-->>B: Données réelles extraites
    B->>C: Prompting + Context Web
    C-->>B: Itinéraire structuré (JSON)
    B-->>F: Pack Voyage Complet
    F-->>U: Rendu visuel & Photos HD
```

Cas d'Usage 2 : Authentification Utilisateur (JWT & Supabase)

```
sequenceDiagram
    participant U as Utilisateur
    participant F as Frontend
    participant B as Backend
    participant S as Supabase Auth

    U->>F: Saisit ses identifiants
    F->>B: POST /api/auth/login
    B->>S: Vérification credentials
    S-->>B: Retourne User & JWT Token
    B-->>F: HTTP 200 + Token
    F-->>U: Redirection vers Dashboard
```

Cas d'Usage 3 : Sauvegarde d'un Voyage (Persistance)

```
sequenceDiagram
    participant U as Utilisateur
    participant F as Frontend
    participant B as Backend
    participant DB as Supabase DB

    U->>F: Clique sur "Sauvegarder"
    F->>B: POST /api/trips (avec JWT)
    B->>B: Validation du Token
    B->>DB: INSERT INTO trips (user_id, pack_data)
    DB-->>B: Confirmation (Record créé)
    B-->>F: Succès (Trip ID retourné)
    F-->>U: Feedback visuel (Notification)
```

5. Spécifications API (Externes & Internes)

API Externes & Justifications

Service	Rôle	Justification du choix
Claude 3.5 Sonnet	LLM Orchestrator	Supériorité dans le respect des schémas JSON stricts et finesse de la conciergerie VIP.
Tavily AI	Web Search Agent	Optimisé pour l'IA, il retourne des données structurées (Vols, Hôtels) sans le "bruit" des moteurs classiques.
Unsplash API	Visual Assets	Accès à une bibliothèque de photos haute résolution libre de droits via recherche sémantique.
Supabase	BaaS (DB & Auth)	Infrastructure PostgreSQL robuste et système d'authentification prêt à l'emploi (Sécurité).

Endpoints de l'API Interne (TripGenie API)

1. Authentification

- **POST** /api/auth/login : { "email": "...", "password": "..." } -> { "user": {...}, "token": "..." }
- **POST** /api/auth/register : { "email": "...", "password": "..." } -> { "message": "Success" }

2. Intelligence Artificielle & Génération

- **POST** /api/ai/generate
 - **Input** : { "prompt": "..." }
 - **Output** : JSON structuré contenant Itinéraire, Hôtels, Vols et Météo.

3. Gestion des Voyages (Persistance)

- **GET** /api/trips : Retourne la liste des voyages sauvegardés de l'utilisateur.
- **POST** /api/trips : Sauvegarde un nouveau pack voyage dans Supabase.

6. Plans SCM & Assurance Qualité (QA)

Gestion du Code Source (SCM)

Le projet utilise **Git** pour le contrôle de version, avec une stratégie de branchement rigoureuse :

- **Branches principales** :
 - **main** : Code stable prêt pour la production.
 - **develop** : Branche d'intégration pour les tests de pré-production.
- **Branches de fonctionnalités** :
 - **feat/*** : Chaque nouvelle tâche est développée sur une branche dédiée (ex: **feat/ai-orchestrator**) avant d'être fusionnée via une Pull Request.
- **Code Review** : Toutes les demandes d'extraction vers la branche **develop** font l'objet d'une revue systématique.

Stratégie d'Assurance Qualité (QA)

La fiabilité du système repose sur une approche de tests multi-niveaux :

- **Tests Unitaires (Vitest)** : Validation isolée de la logique métier (calculs de budget, formatage des données IA).
- **Tests d'API (Postman)** : Utilisation de collections Postman pour vérifier la conformité des réponses JSON.
- **Tests Manuels** : Validation systématique des flux critiques (User Journey) : Connexion -> Chat -> Génération -> Sauvegarde.

Pipeline de Déploiement (CI/CD)

1. **Local** : Développement et tests unitaires.
 2. **Staging** : Déploiement automatique de `develop` sur un environnement de test.
 3. **Production** : Déploiement de `main` après validation finale.
-

7. Justifications Techniques & Décisions d'Architecture

- **Tavily AI (Recherche Agentique)** : Choisi pour résoudre le problème majeur des LLM : l'obsolescence des données. En couplant Claude 3.5 à Tavily, le système accède à des prix et disponibilités en temps réel, garantissant la fiabilité du MVP.
- **Claude 3.5 (Orchestration)** : Ce modèle a été retenu pour sa supériorité dans le "Reasoning" et sa capacité à produire du code JSON structuré sans erreurs, indispensable pour la stabilité du Frontend.
- **Supabase (Infrastructure PostgreSQL)** :
 - **Choix stratégique** : Plutôt que de consacrer du temps à la maintenance d'un serveur DB Linux, le choix s'est porté sur une solution managée pour garantir une sécurité maximale (Authentification JWT, Chiffrement des données au repos).
 - **Respect des fondamentaux** : Bien que le service soit managé, la **conception relationnelle** (MCD/MLD) a été entièrement réalisée à la main (Schéma ERD). L'utilisation de PostgreSQL standard garantit la portabilité totale du projet vers n'importe quel autre hébergeur SQL si nécessaire.
 - **Sécurité** : L'utilisation de Supabase Auth permet de respecter les standards de sécurité actuels (OAuth2, Sessions sécurisées), une priorité absolue pour une application gérant des itinéraires personnels.